

BruFace

MA1 Mechanical Engineering
Vibrations and Acoustics
Academic year 2022-2023

Session 2 : Multiple DOF system

Welcome to the second session

During the practical session, we will use the same libraries as the one used in the session 1.

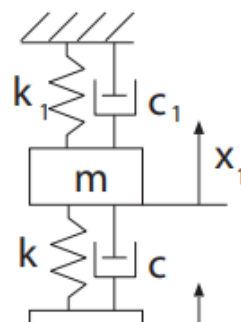
Don't forget :It is important that you run each cell of the notebook. To do so select a cell (it will be highlighted) and press shift+enter on your keyboard or the play button in the menu above.

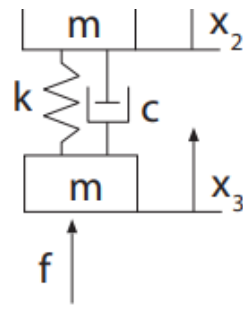
```
In [2]: # let's import both numpy and matplotlib.pyplot
# the "as" in the code below is to tell python that if we reference
import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as linalg
# don't worry about this line
%matplotlib notebook
```

Exercises

Exercise 1: Fundamental equations of a MDOF system

Consider the following three-degree-of-freedom (3 DOFs) system :





Assignment 1.1: Write the equation of motion in the time domain for the system (in LaTeX) in the cell below:

(If you are not familiar with LaTeX you can use: <https://www.hostmath.com/> and copy the LaTeX below, or write it on paper for yourself)

Type *Markdown* and LaTeX: α^2

Assignment 1.2: Create a function `build_3DOF_matrices` that takes as arguments the stiffnesses k and k_1 , the damping values c and c_1 and the mass m and computes the system matrices (M , C , K). In the cell below we have given you a start, we show how you can use `np.array` to build the K -matrix.

```
In [3]: def build_3DOF_matrices(m:float,c:float,c1:float,k:float,k1:float):
        """
        Implement a function that computes the build the system matrix for
        a 3DOF system.

        Arguments:
        m,k,k1,c,c1 -- parameters of the MDOF system

        Returns:
        M_mat : The M matrix from the system equation
        K_mat : The K matrix from the system equation
        C_mat : The C matrix from the system equation

        Tips use np.array([[ ],[ ],[ ]]) to define your 2D matrix
        check documentation FMI 'Numpy.array()'
        """
        # here is an example of how to set a matrix

        M_mat=np.array([[m, 0, 0],
                        [0, m, 0],
                        [0, 0, m]])

        ###BEGIN SOLUTION

        K_mat=np.array([[k1+k, -k, 0],
                        [-k, 2*k, -k],
                        [0, -k, k]])
        C_mat=np.array([[c1+c, -c, 0],
                        [-c, 2*c, -c],
                        [0, -c, c]])
        ###END SOLUTION
        return M_mat, C_mat, K_mat
```

In the cell below we will check if you implemented the function correctly. To do so we will use several properties of the system matrices. Do you understand all the checks that are being done?

```
In [4]: M, C, K = build_3DOF_matrices(m=1, k=10, k1=10, c=0.1, c1=0.1)

assert np.array_equal(M, np.eye(3)), 'M-matrix is not diagonal' # Mass matrix

for matrix in [M,C,K]: # Iterate over the three matrices
    assert np.size(matrix,0) == np.size(matrix, 1), f'{matrix} is not square'
    assert np.size(matrix,0) == 3, f'{matrix} is not 3x3' # For a 3DOF system
    assert np.allclose(matrix, matrix.T), f'{matrix} is not symmetric'
    # The next one is a bit special, we assert whether or not the matrix is positive definite
    assert np.all(np.linalg.eigvals(matrix) > 0), f'{matrix} is not positive definite'

# Some final checks
assert K[2,2] == 10
assert np.sum(K) == 10, 'The K matrix has an error in it'
assert np.sum(C) == 0.1, 'The C matrix has an error in it'
```

Exercise 2 : Solving the conservative system

Before diving into a damped response we first will analyze the **conservative** system response. Let us start with the eigenvalues and eigenvectors of the conservative system. To do so, we need to solve the following equation:

$$(K - \lambda M)\Phi = (K - \omega^2 M)\Phi = 0$$

Which is a basic eigenvalue problem. In the cell below we provide you a function `calc_eigenvalues_eigenvectors` that uses `scipy.linalg.eig` to solve the eigenvalue problem and returns λ and Φ .

```
In [5]: def calc_eigenvalues_eigenvectors(M:np.array,K:np.array):
    """
    A function that computes the eigenvalues and eigenvectors
    based on the system parameter by reworking the equations of motion

    Arguments:
    M,K -- system matrices of the MDOF system

    Returns:
    lamb -- The eigenvalues of the conservative eigenvalue problem
    phi -- The eigenvectors of the conservative eigenvalue problem

    """

    lamb, phi = linalg.eig(K,M)

    return lamb, phi
```

Assignment 2.1: Use `build_3DOF_matrices` and `calc_eigenvalues_eigenvectors` to first calculate the eigenvalues `lamb` and

eigenvectors `phi` (a.k.a the mode shapes). Then calculate the eigenfrequencies in Hz for a 3DOF system as presented above with $m = 1kg$, $k = k1 = 16N/m$, $c = c1 = 0.1Ns/m$.

Put your found eigenfrequencies in an array called `f_res`.

```
In [13]: ###BEGIN SOLUTION

M, C, K = build_3DOF_matrices(m=1, k=16, k1=16, c=0.1, c1=0.1)
lamb, phi = calc_eigenvalues_eigenvectors(M,K)

f_res = np.sqrt(lamb)/2/np.pi

###END SOLUTION
```

Let us have a look at the resonance frequencies you calculated:

```
In [14]: # Enumerate() method adds a counter to the list f_res and returns it
for i,f_r in enumerate(f_res):
    print(f'Resonance frequency {i}: {np.real(f_r):0.2f} Hz')

# I do a little test, summing up the found resonance frequencies and
assert len(f_res) == 3, f'Your result contains {len(f_res)} resonance
assert round(np.sum(np.real(f_res)),2) != 12.73, 'Did you spot the p
assert round(np.sum(np.real(f_res)),2) == 2.22, 'At least one of the
```

```
Resonance frequency 0: 1.15 Hz
Resonance frequency 1: 0.79 Hz
Resonance frequency 2: 0.28 Hz
```

Assignment 2.2: In the field below answer following questions:

- How many resonance frequencies did you obtain? Why?
- How many mode shapes did you obtain? Why?
- What are the dimensions of the mode shapes? Why?

In []:

An important property of mode shapes is their orthogonality. In the next assignment you have to validate the orthogonality between two mode shapes.

Assignment 2.3: Complete `check_orthogonality` to calculate the orthogonality between two vectors. Consider using `np.matmul` and `np.transpose`:

TODO: This exercise is slightly wrong the orthogonality is only with respect to K or M so that has also be inputted here!!!

```
In [11]: def check_orthogonality(phi_1, phi_2):
        """
        A function that computes the orthogonality between two vectors

        Arguments:
        phi_1, phi_2 -- vectors of which the orthogonality is checked

        Return: orth -- the orthogonality of the two vectors

        TIP: matrix multiplications are done using np.matmul
        """
        ###BEGIN SOLUTION

        orth = np.matmul(np.transpose(phi_1), phi_2)
        ###END SOLUTION

        return orth
```

Let us check whether your function does what it must do:

```
In [12]: # As a test let us take two orthogonal vectors
vec_1 = np.array([1,1,0])
vec_2 = np.array([1,-1,0])

assert np.abs(check_orthogonality(vec_1, 5*vec_1)) > 1.e-10 # not equal to zero
assert np.isclose(check_orthogonality(vec_1, vec_2), 0) # = zero up to machine precision
assert np.isclose(check_orthogonality(vec_1, 5*vec_2), 0) # = zero up to machine precision
```

Assignment 2.4: Use `check_orthogonality` to check the orthogonality between your modeshapes:

```
In [20]: ###BEGIN SOLUTION

results = np.empty((3,3))
i = 0
j = 0

for phi_i in phi.T:
    for phi_j in phi.T:
        results[i,j] = round(np.abs(check_orthogonality(phi_i, phi_j)))
        j+=1
    i+=1
    j=0

results
###END SOLUTION
```

```
Out[20]: array([[1., 0., 0.],
               [0., 1., 0.],
               [0., 0., 1.]])
```

Were your mode shapes nicely orthogonal?

Exercise 3 : The frequency response function

Let us continue with the 3DOF freedom model we used in Exercise 1. This time we will focus on the frequency response function. The frequency response function is a way to represent how a system will respond to a force being exerted on any of the degrees of freedom. For this we start with standard equations of motion of a MDOF system expressed in the frequency domain.

$$(-\omega^2 M + j\omega C + K) \vec{X}(\omega) = \vec{F}(\omega)$$

Which we can rewrite as follows:

$$\vec{X}(\omega) = (-\omega^2 M + j\omega C + K)^{-1} \vec{F}(\omega) = H(\omega) \vec{F}(\omega)$$

In which we call $H(\omega)$ the frequency response function.

Which we can rewrite as follows:

$$\vec{X}(\omega) = (-\omega^2 M + j\omega C + K)^{-1} \vec{F}(\omega) = H(\omega) \vec{F}(\omega)$$

In which we call $H(\omega)$ the frequency response function.

In the cell below the function `frequency_response_function` calculates the FRF for a system defined using M , C and K matrices for a range of frequencies.

```
In [16]: def frequency_response_function(M:np.array, C:np.array, K:np.array, f):
    """
    A function that computes the Frequency Response Function from the
    Arguments:
    M,C,K -- system matrices of the MDOF system
    f -- the frequency for which the response needs to be calculated
    i,j -- the specific element of the FRF matrix to be computed.

    Return: H_ij -- the i-th row and j-th column element of the FRF matrix
    """

    if not isinstance(f, np.ndarray):
        f = np.array([f])
    s = f*np.pi*2*1j

    M_s2 = np.outer(M, (s**2)).reshape(M.shape[0], -1, len(s))
    C_s = np.outer(C, (s**1)).reshape(M.shape[0], -1, len(s))
    K_s = np.outer(K, (s**0)).reshape(M.shape[0], -1, len(s))

    H_ij = []
    for s_i in range(len(s)):
        H_ij.append(np.linalg.inv((M_s2+C_s+K_s)[:,:,:s_i])[i,j])

    return H_ij
```

Assignment 3.1: In the field below answer following questions:

- What are the dimensions of the FRF for a 3DOF system?
- Can you explain in your own words what the FRF represents?

In []:

Assignment 3.2: For the 3DOF system of exercise 1, can you calculate the transfer function between the input location of the force f and the output location x_3 for a range of frequencies between 0 and 2Hz using `frequency_response_function` ? Call your frequency vector `f` and your output `H_3f`

TIP: Be aware that in python counting starts from 0, so the first column of a matrix is denoted 0

TODO: This exercise does not enforce that they are actually plotting H_{33} , a lot of students got past this segment without actually plotting H_{33} !!

```
In [17]: ###BEGIN SOLUTION
f= np.linspace(0,2,200)
M, C, K = build_3DOF_matrices(m=1, k=16, k1=16, c=0.1, c1=0.1)
H_3f = frequency_response_function(M, C, K, f= f, i=2,j=2)
###END SOLUTION
```

Let us have a look at your output:

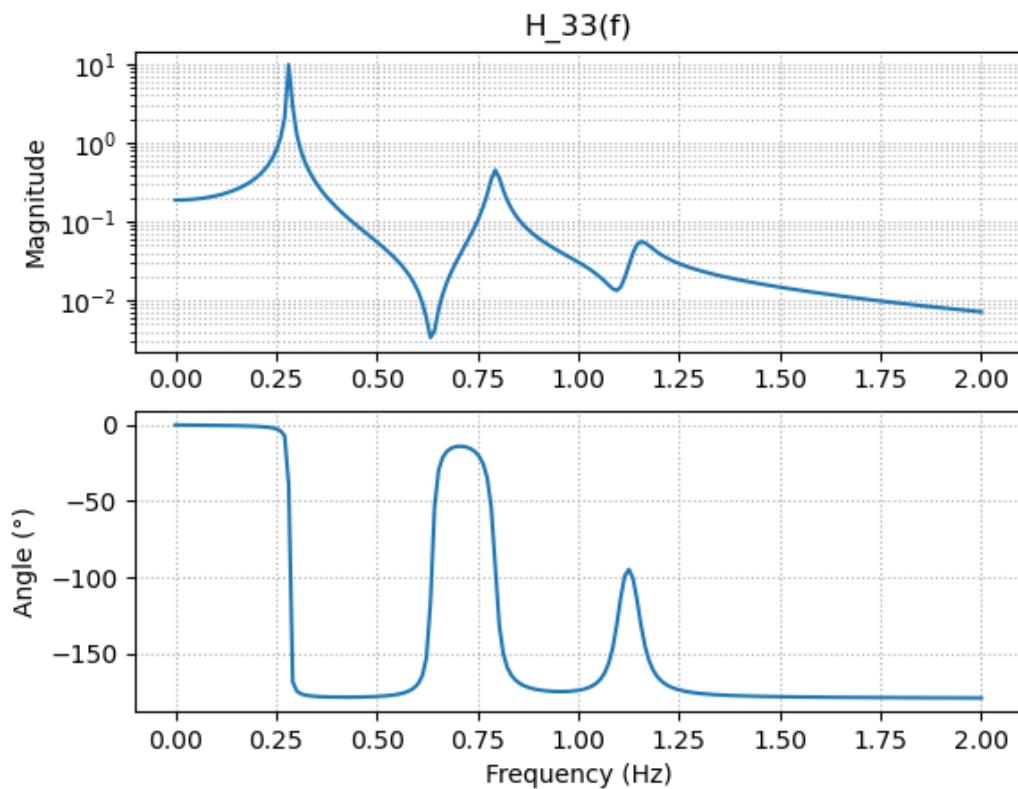
```
In [18]: fig, axes = plt.subplots(2,1)

axes[0].semilogy(f,np.abs(H_3f))
axes[0].grid(which='both', linestyle=':')
axes[0].set_ylabel('Magnitude')
axes[1].plot(f,np.angle(H_3f,deg=True))
axes[1].set_ylabel('Angle (°)')
axes[1].set_xlabel('Frequency (Hz)')

axes[1].grid(which='both', linestyle=':')

t = axes[0].set_title('H_33(f)')
```

<IPython.core.display.Javascript object>



Obviously the properties of the FRF will change depending on the system properties. In the following interactive cell you can change the values of m , c , k . Can you identify the role of each of the parameters on the final FRF?

```

In [19]: from ipywidgets import interact, fixed
from matplotlib.widgets import Slider, Button

# Define initial parameters
init_m = 1
init_k = 16
init_c = 0.1

init_M, init_C, init_K = build_3DOF_matrices(m=init_m, c1=init_c, c2=init_c)

# Create the figure and the line that we will manipulate
fig, axes = plt.subplots(2,1)

f = np.arange(0,2,0.01)
H = frequency_response_function(M=init_M,K=init_K,C=init_C,f=f,i=2,j=2)

# wn=calcW_n(k=init_k,m=init_m)
axes[0].semilogy(f,np.abs(H), lw=2, color='lightgray')

line, = axes[0].semilogy(f,np.abs(H), lw=2, color='steelblue')

axes[0].set_xlabel('Frequency (Hz)')
axes[0].set_ylabel('|H(f)|')
axes[0].grid(which='both', linestyle=':')

axes[1].plot(f,np.angle(H,deg=True), color='lightgray')

line2, = axes[1].plot(f,np.angle(H,deg=True), color='steelblue')

axes[1].set_xlabel('Frequency (Hz)')
axes[1].set_ylabel('angle(H(f))')
axes[1].set_ylim([-180,0])

axes[1].grid(which='both', linestyle=':')

# adjust the main plot to make room for the sliders
plt.subplots_adjust(left=0.25, bottom=0.4)

axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
m_slider = Slider(
    ax=axmass,
    label='Mass (kg)',
    valmin=0.1,
    valmax=10,
    valinit=init_m,
)

axstiffness=plt.axes([0.25, 0.15, 0.65, 0.03])
k_slider = Slider(
    ax=axstiffness,
    label="Stiffness (N/m)",
    valmin=1,
    valmax=20,
    valinit=init_k,
)

```

```

axc = plt.axes([0.25, 0.2, 0.65, 0.03])
c_slider = Slider(
    ax=axc,
    label="friction (N/(m*s))",
    valmin=0.001,
    valmax=1,
    valinit=init_c,
)

# The function to be called anytime a slider's value changes
def update(fixed):

    M_u, C_u, K_u = build_3DOF_matrices(
        m=m_slider.val,
        c1=c_slider.val,
        c=c_slider.val,
        k=k_slider.val,
        k1=k_slider.val
    )

    H = frequency_response_function(M_u, C_u, K_u, f=f, i=2, j=2)

    line.set_ydata(np.abs(H))
    line2.set_ydata(np.angle(H, deg=True))

    fig.canvas.draw_idle()

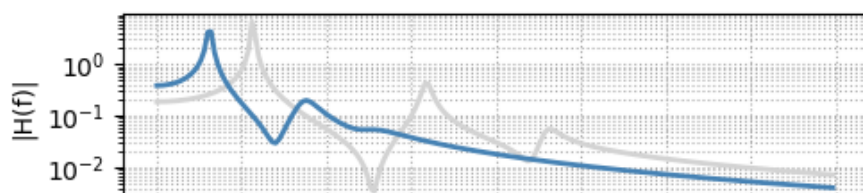
# register the update function with each slider
m_slider.on_changed(update)
k_slider.on_changed(update)
c_slider.on_changed(update)

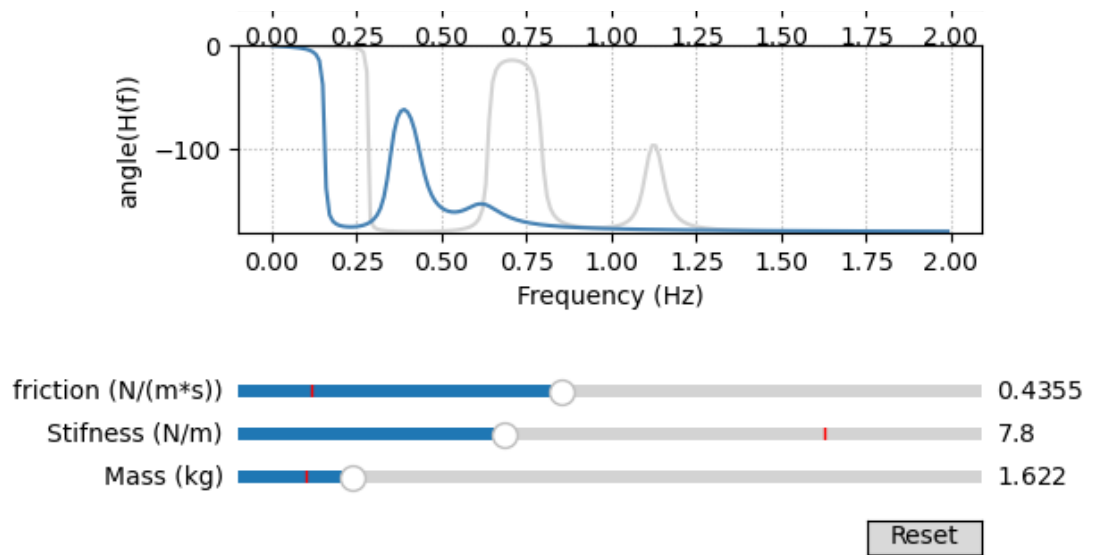
# Create a `matplotlib.widgets.Button` to reset the sliders to initial
resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    m_slider.reset()
    k_slider.reset()
    c_slider.reset()
    button.on_clicked(reset)

w = interact(update(fixed))
<IPython.core.display.Javascript object>

```





As discussed in assignment 3.1 the FRF is in fact a 3 by 3 matrix for a three dimensional system. Up to this point we just focused on the transfer function between input location f and output location $x_3(t)$. But how do the other elements of the transfer function look like?

Assignment 3.3: Update the code below to plot the FRF function for all 9 elements. Can you also plot the resonance frequencies on the x-axis?

TIP : To plot a figure with logarithmic y-scale on a particular axis `ax` you can use the command `ax.semilogy` and to draw a single vertical line the same axis you can use the command `ax.vline(value)`.

TODO: Review this exercise, a lot of students did not understand properly

```

In [21]: f = np.linspace(0,2,200)
n_dof = np.shape(M)[0]

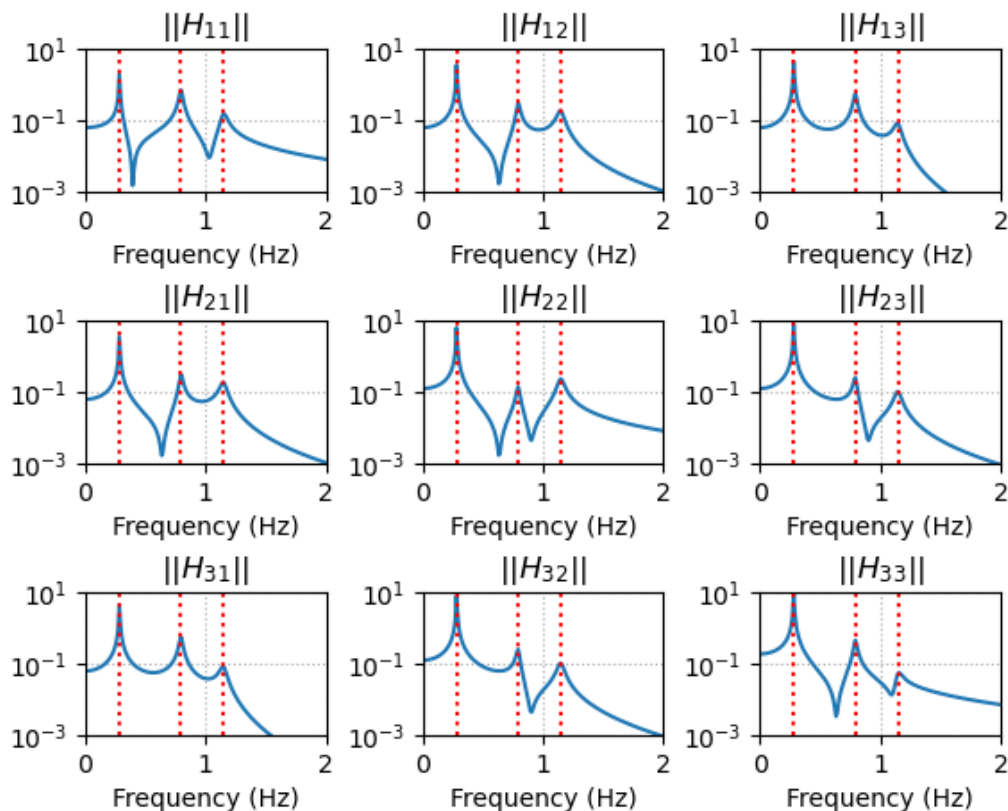
fig, axes = plt.subplots(n_dof,n_dof)

for i in range(n_dof):
    for j in range(n_dof):
        ###BEGIN SOLUTION
        axes[i,j].semilogy(f,np.abs(frequency_response_function(M,C,f)))
        for fr in f_res:
            axes[i,j].axvline(np.real(fr), linestyle=':', color='r')
        ###END SOLUTION
        axes[i,j].set_title(f'$||H_{{{i+1}}}{{{j+1}}}$')
        axes[i,j].set_xlabel('Frequency (Hz)')
        axes[i,j].grid(which='both', linestyle=':')
        axes[i,j].set_ylim([1e-3,1e1])
        axes[i,j].set_xlim([0, max(f)])

plt.subplots_adjust(left=0.1,
                    bottom=0.1,
                    right=0.9,
                    top=0.9,
                    wspace=0.4,
                    hspace=0.9)

```

<IPython.core.display.Javascript object>



Assignment 3.4: Can you answer following questions about the FRF's in the cell below?

- Do you notice any difference between the FRF's on the diagonal elements and those on non-diagonal elements?
- What do you notice about the resonance frequencies?

- What can you tell about the zeroes of the system?
- Can you spot other properties of the frequency response function?

In []:

TODO: Stop the session here.

Exercise 4: Projecting on the modal space

In this exercise we'll use the real mode shapes (obtained from solving the conservative eigenvalue problem) to project our equations of motion onto the modal basis.

Assignment 4.1: Complete `project_modal` to project a system matrix into the modal space using the matrix of eigenvectors `phi` :

TIP: You might need to use `np.matmul` multiple times

```
In [22]: def project_modal(A, phi):
          """
          A function that projects the matrix A onto the modal space using

          Arguments:
          A -- Matrix to be projected into the modal space, can be M, C or
          phi -- Matrix of real mode shapes

          Return: A_prj -- the projection of A onto the modal space
          """
          ###BEGIN SOLUTION

          A_prj = np.matmul(np.matmul(np.transpose(phi), A), phi)

          ###END SOLUTION

          return A_prj
```

Assignment 4.2: Let us test your implementation of `project_modal` by again taking the system from Exercise 1 with $m = 1\text{kg}$, $k = k_1 = 16\text{N/m}$, $c = c_1 = 0.1\text{Ns/m}$. And calculating the eigenvalues and eigenvectors for the conservative system. Project the M, C and K -matrices onto the modal space. Call these projections `M_prj`, `C_prj` and `K_prj` respectively.

```
In [23]: ###BEGIN SOLUTION
M, C, K = build_3DOF_matrices(m=1, k=16, k1=16, c=0.1, c1=0.1)
lamb, phi = calc_eigenvalues_eigenvectors(M, K)

M_prj = project_modal(M, phi)
K_prj = project_modal(K, phi)
C_prj = project_modal(C, phi)

###END SOLUTION
```

What do you expect M_{prj} , K_{prj} and C_{prj} to look like? Write your answer below:

Type *Markdown* and LaTeX: α^2

Let us check if your results are correct, to do so we use your observation above

```
In [24]: n_dof = np.size(M_prj,0) # Get the number of DOF's

for matrix in [M_prj, C_prj, K_prj]: # Iterate over the three matrices
    for i in range(n_dof):
        for j in range(n_dof):
            if i != j:
                assert np.isclose(matrix[i,j],0) # What are we checking?
```

Assignment 4.3: All three matrices M, C, K can be projected successfully on the modal space. This is no surprise for M and K . But why can we successfully project C onto the modal space? What will happen when we change the value of c_1 ? Write your answer below.

Type *Markdown* and LaTeX: α^2

Now that we have successfully projected our results into the modal space we have projected our 3DOF system onto 3 SDOF systems.

For our next assignment we will rely on our work from session 1. Therefore we will import the function `impulse_response` that we defined in session 1. You'll notice that we will import this one first.

Assignment 4.4: Complete the cell below using `impulse_response` to calculate the impulse responses $h_i(t)$ of the 3 SDOF systems that were obtained from the modal projection. Combine all three responses into a list called `impulse_responses`

TIP If you are not sure how to use `impulse_response` you can always use `?impulse_response` the documentation will appear, providing some help.

```
In [25]: from session1 import impulse_response # Importing from the previous session

f_s = 10 # Defining a sampling frequency
t = np.linspace(0,600, f_s*600) # Defining a time vector of 600s, sampled at 10Hz

impulse_responses = [] # to add new elements to this list you can use .append()

###BEGIN SOLUTION
for i in range(3):
    impulse_responses.append(impulse_response(m=M_prj[i,i],c=C_prj[i,i]))
###END SOLUTION
```

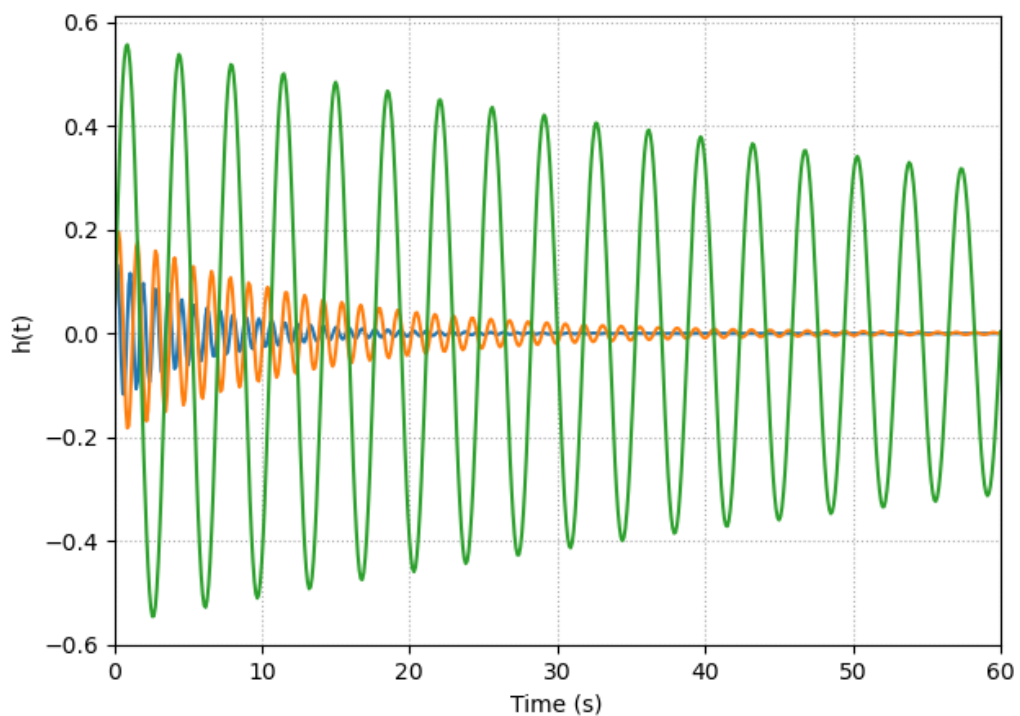
Let us perform some assertions to check your results and then plot them.

```
In [26]: # Doing some checks
assert len(impulse_responses) == 3, "impulse_responses does not contain 3 elements"
assert not np.array_equal(impulse_responses[0], impulse_responses[1])
for h_i in impulse_responses:
    assert len(h_i) == len(t), "The length of your calculated modal impulse response must be equal to the length of the time vector"

# actual plotting
plt.figure(figsize=(7,5))

for h_i in impulse_responses:
    plt.plot(t, h_i)
plt.xlim([0,60])
plt.xlabel('Time (s)')
plt.ylabel('h(t)')
plt.grid(which='both', linestyle=':')
plt.show()
```

<IPython.core.display.Javascript object>



The total impulse response of the system is the sum of these three SDOF impulse responses. The only element missing is the spatial information that is contained in the mode shapes. To illustrate this we will use the modal impulse responses and the mode shapes to simulate the response of the bottom DOF (x_3) when a force is applied at the same DOF (as illustrated in the drawing in Exercise 1).

As a force signal we will use a swept sine also known as a sine sweep or chirp. To do so we will use the command `chirp` from the `scipy.signal` package.

Assignment 4.4: Use `chirp` to generate a swept sine signal that ranges from 0 to 2 Hz, amplitude equal to 1 and have it start at a value of 0. Reuse the timevector `t` that was introduced in the previous cells. Name the signal `f_chirp`.

```
In [27]: from scipy.signal import chirp

        ###BEGIN SOLUTION
        f_chirp = chirp(t, 0, max(t), 2, phi=90)
        ###END SOLUTION
```

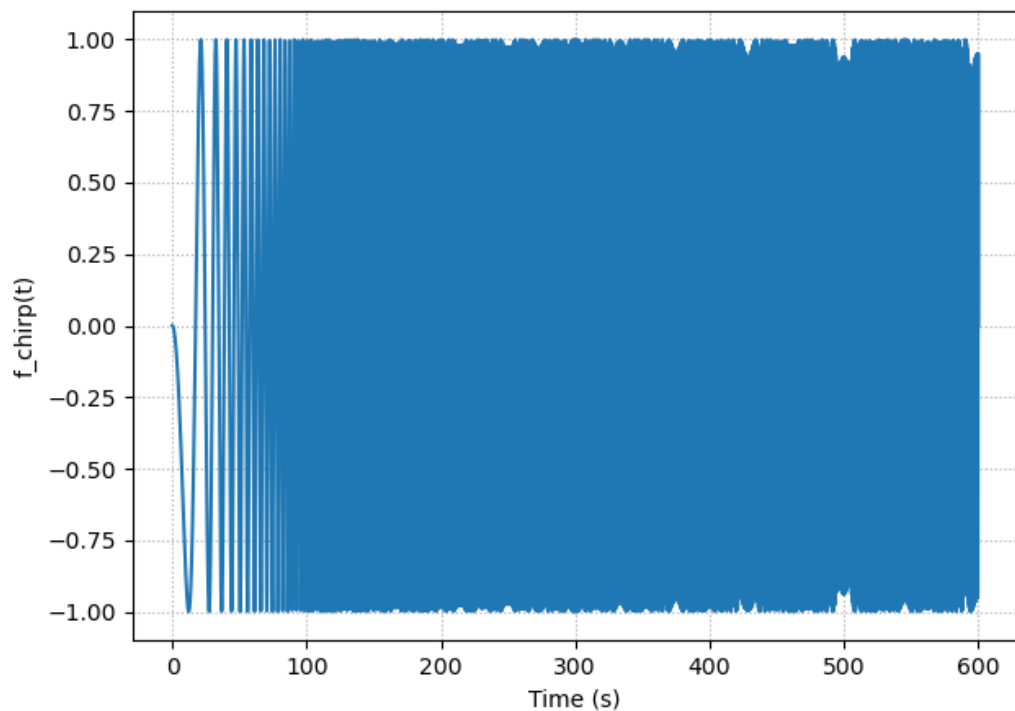
Let us check some of your results and plot the swept sine signal.

```
In [28]: assert len(f_chirp) == len(t), 'The length of the chirp signal does
        assert np.isclose(f_chirp[0], 0), 'Your swept sine does not start at
        assert max(f_chirp) <= 1, 'The amplitude of your swept sine is too la

        # Plotting the results

        plt.figure(figsize=(7,5))
        plt.plot(t,f_chirp)
        plt.xlabel('Time (s)')
        plt.ylabel('f_chirp(t)')
        plt.grid(which='both', linestyle=':')
        plt.show()
```

<IPython.core.display.Javascript object>



Let us now apply this force to the bottom DOF (x_3). To do so we must first calculate the modal excitation $f_i(t)$ at this location (for each mode).

Assignment 4.5: Finish `modal_excitation` to calculate the modal excitation for the chirp signal.

TIP: never forget python starts counting from 0

```
In [29]: def modal_excitation(phi, i, input_location, f_input, n_dof=3):
        """
        A function that calculates the modal excitation f_i using the real mode shapes
        and considering a single input location

        Arguments:
        phi -- Matrix of real mode shapes
        i -- mode to be considered
        input_location -- the input DOF (or force location)
        f_input -- timeseries of the force used as excitation to the system
        n_dof -- Number of DOF's, defaults to 3

        Return: f_i -- the modal excitation
        """

        f_loc = np.zeros([n_dof,1]) # I recommend using a vector to identify the input location
        #Set the element associated with the input location to 1

        ###BEGIN SOLUTION
        f_loc[input_location] = 1
        f_si = np.matmul(np.transpose(phi[:,i]),f_loc)
        f_i = f_si * f_input
        ###END SOLUTION

        return f_i
```

Let us check your function; by using some simple checks for a second order system.

```
In [30]: phi_2dof = np.array([[1,1],[1,-1]]) # Textbook mode shapes of a 2DOF system

assert np.isclose(modal_excitation(phi_2dof, 0, 1, 1, n_dof=2),1), 'Should be 1'
assert np.isclose(modal_excitation(phi_2dof, 1, 1, 3, n_dof=2),-3), 'Should be -3'
assert np.isclose(modal_excitation(phi_2dof, 0, 0, 2, n_dof=2),2), 'Should be 2'
assert np.isclose(modal_excitation(phi_2dof, 1, 0, 2, n_dof=2),2), 'Should be 2'
```

Now that we can calculate the modal excitations $f_i(t)$ for each of the three modes. Let us calculate the modal responses for a mode i at the 3rd DOF: $x_{3,i}(t)$. Remember that

$$\vec{x}_i(t) = z_i(t)\vec{\psi}_i$$

in which $z_i(t)$ is the SDOF response to the modal force:

$$z_i(t) = h_i(t) * f_i(t)$$

Assignment 4.6: Finish `output_modal_response` to calculate the modal response x_i of to a modal excitation f_i .

```
In [31]: def output_modal_response(phi, i, h_i, f_i, output_location):

    """
    A function that calculates the modal response ( $x_i(t)$ )

    Arguments:
    phi -- Matrix of real mode shapes
    i -- mode to be considered
    h_i -- impulse response of the i-th mode
    f_i -- timeseries of the modal excitation under the i-th mode
    output_location -- the output DOF

    Return: x_i -- the modal response at the given output location
    """

    ###BEGIN SOLUTION
    z_i = np.convolve(h_i, f_i, mode='full')
    x_i = z_i * phi[output_location,i]
    ###END SOLUTION

    return x_i
```

And let us again check this function, again using some standard behaviour of a 2DOF system.

```
In [32]: h_i = impulse_responses[0]
assert np.array_equal(h_i*-1, output_modal_response(phi_2dof, 1, h_i,
assert np.array_equal(h_i*2, output_modal_response(phi_2dof, 0, h_i,
```

Let us see what we can do with your functions. Let us plot the modal responses of the system of Exercise 1 to the forces we calculated.

In the figure below the individual responses $x_i(t)$ are plotted. Do they make sense to you?

```

In [33]: output_location = 2 # this is x3
input_location = 2 # The force is applied at the same location

plt.figure(figsize=(7,5))

x_i_list= []
for h_i, i in zip(impulse_responses, range(len(impulse_responses))):

    f_i = modal_excitation(phi, i, input_location, f_chirp)
    x_i = output_modal_response(phi, i, h_i, f_i, output_location)

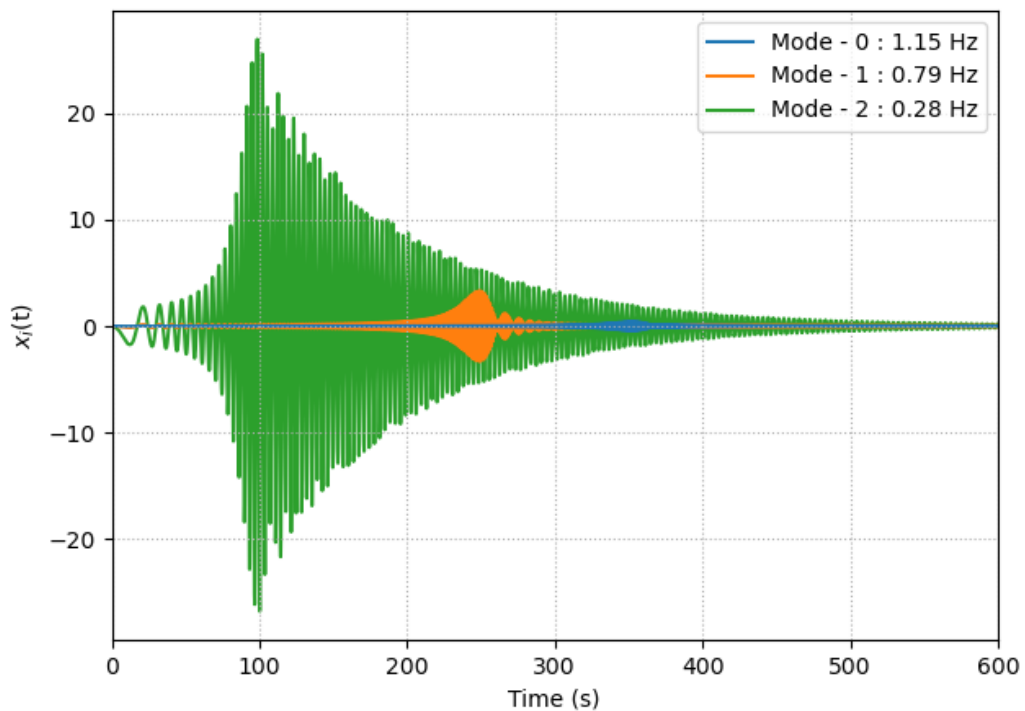
    x_i_list.append(x_i[:len(t)])

    plt.plot(t, x_i[:len(t)], zorder=-i, label=f'Mode - {i} : {np.rec

plt.xlabel('Time (s)')
plt.ylabel('$x_i(t)$')
plt.xlim([0,600])
plt.legend()
plt.grid(which='both', linestyle=':')
plt.show()

```

<IPython.core.display.Javascript object>



Assignment 4.7: Can you explain the behaviour observed above in the cell below

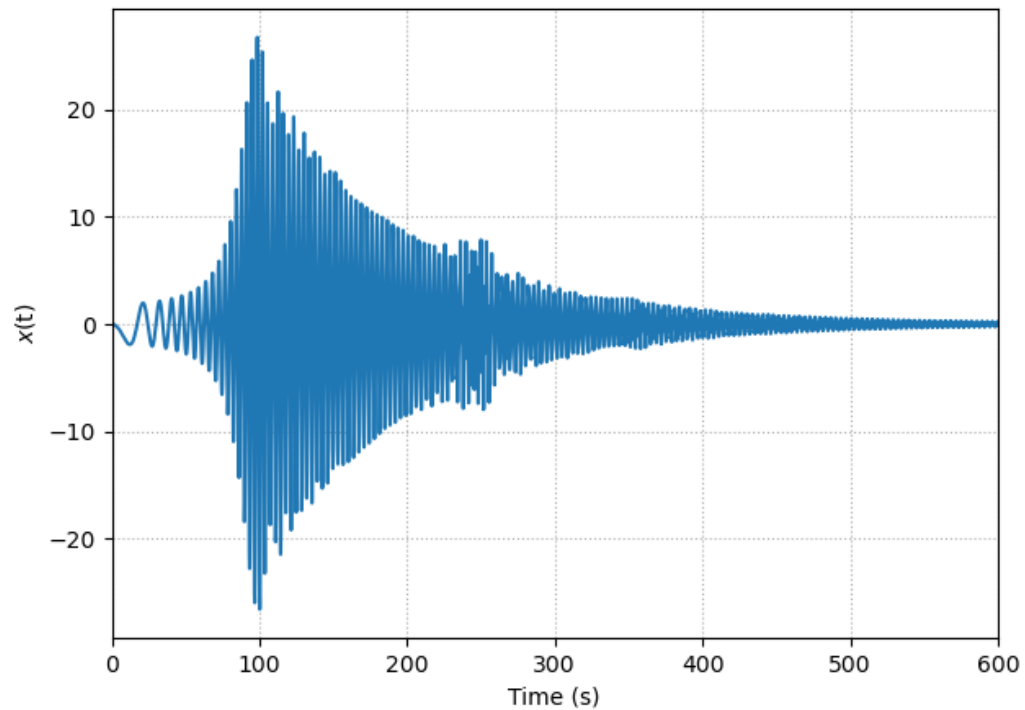
Type *Markdown* and LaTeX: α^2

The total response of the system to the chirp signal is simply the sum of all three $x_i(t)$.

```
In [34]: x = sum(x_i_list)

plt.figure(figsize=(7,5))
plt.plot(t, x[:len(t)])
plt.xlabel('Time (s)')
plt.ylabel('$x(t)$')
plt.xlim([0,600])
plt.grid(which='both', linestyle=':')
plt.show()
```

<IPython.core.display.Javascript object>



Now let us have a look at the spectrum of this response, we clearly see how the 3DOFs contribute to the global response.

```

In [35]: f = np.fft.fftfreq(t.shape[-1],1/f_s)
X = np.fft.fft(x)
F_chirp = np.fft.fft(f_chirp)

fh = int(len(f)/2)

fig, ax1 = plt.subplots()

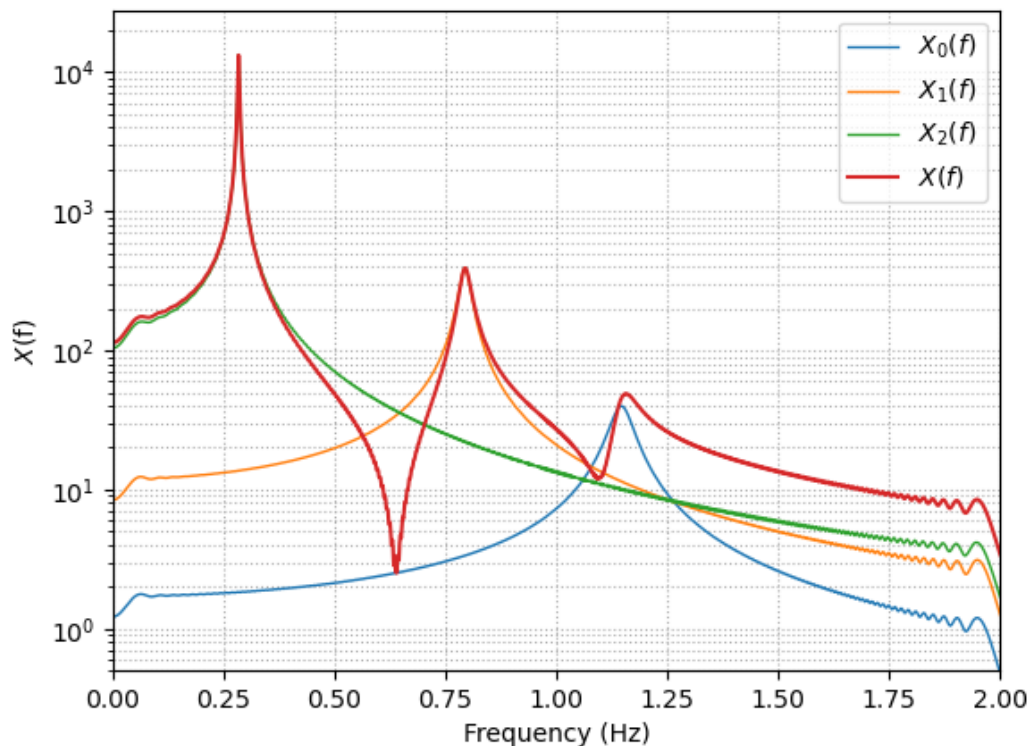
i = 0
for x_i in x_i_list:
    Xi = np.fft.fft(x_i)
    ax1.semilogy(f[:fh], abs(Xi[:fh]), label=f'$X_{i}(f)$', linewidth=1)
    i+=1

ax1.semilogy(f[:fh], abs(X[:fh]), label='$X(f)$')

plt.xlabel('Frequency (Hz)')
plt.ylabel('$X(f)$')
plt.xlim([0,2])
plt.ylim(bottom= 5e-1)
plt.grid(which='both', linestyle=':')
l = plt.legend()

```

<IPython.core.display.Javascript object>

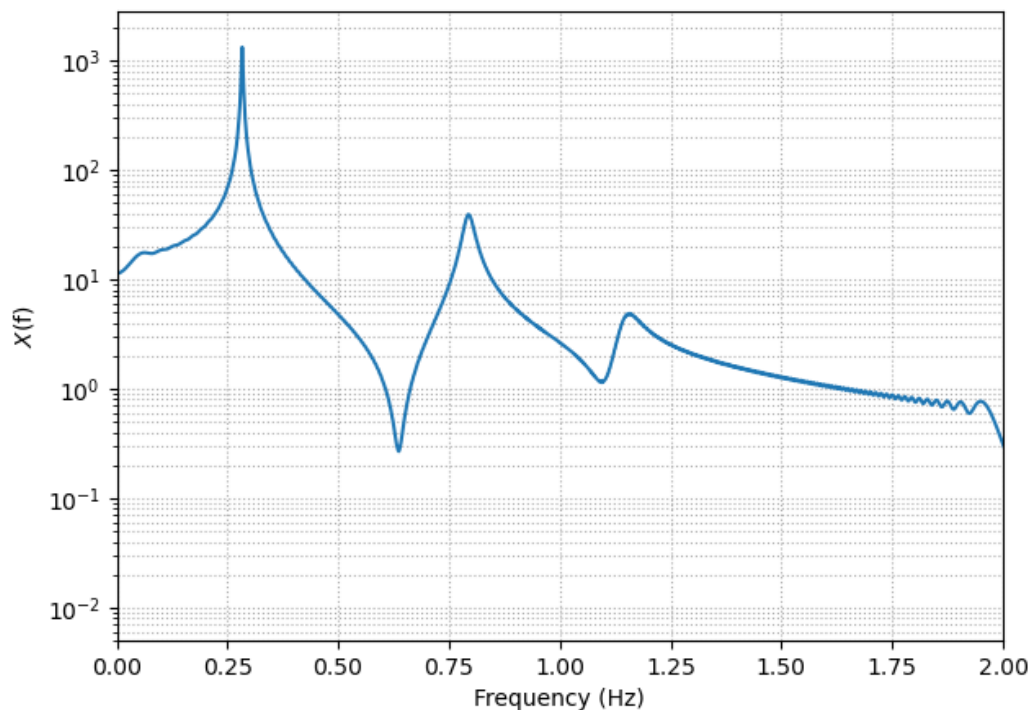


In fact we also obtain a similar result in the frequency domain. For this we use the frequency response function. We simply need to compute the FFT of the original input signal and multiply it with the FRF between the input location and output location.

```
In [36]: f = np.fft.fftfreq(t.shape[-1],1/f_s)
F_T = np.fft.fft(f_chirp) # Transforming the Chirp signal to the frequency domain
X_f = frequency_response_function(M,C,K,f=f, i=output_location,j=input_location)

plt.figure(figsize=(7,5))
plt.semilogy(f, abs(X_f))
plt.xlabel('Frequency (Hz)')
plt.ylabel('$X(f)$')
plt.xlim([0,2])
plt.ylim(bottom= 5e-3)
plt.grid(which='both', linestyle=':')
```

<IPython.core.display.Javascript object>



That concludes this session: In this session we tried to show you how MDOF systems can be modelled and its modal properties can be calculated, the FRF functions can be computed from the system matrices and finally how through the modal projection we can simulate the response of an MDOF system through the superposition of SDOF systems.